Department of Computer Science

Faculty of Engineering, Built Environment & IT

University of Pretoria

COS 301: Software Engineering

Software Requirements Specification

Capstone Project 2026 - Demo 3

Team Name: OmniTech

September 3, 2026

Contents

1 Introduction

Driving Tracker is an intelligent Android-based driving companion developed by OmniTech. The system aims to improve road safety, fuel efficiency, and vehicle maintenance by combining mobile sensors, GPS tracking, and vehicle diagnostics into a single integrated platform. Unlike existing solutions that focus only on navigation or diagnostics, Driving Tracker provides drivers with real-time feedback on driving behavior and vehicle health.

This document explores the software requirements and design specifications of the Driving Tracker application.

2 Project Owner

The project owner and client of the Driving Tracker is Gendac, and our industry mentors are Kevin Cowdrey, Sthabiso Jaffar, and Brandon Craytor.

3 User Stories

3.1 User Management:

1. As a new user, I want to create an account using my email and password so that I can securely access the Driving Tracker platform.
2. As a registered user, I want to securely log into the application so that I can access my driving data and features.
3. As a user, I want to reset my password if I forget it so that I can regain access to my account.
4. As a user, I want to update my profile information so that my account details remain accurate.
5. As a user, I want to add trusted emergency contacts so that they can be notified during dangerous situations.

3.2 GPS Tracking and Trips:

1. As a driver, I want my trips and routes to be tracked in real time so that I can review my journeys later.
2. As a driver, I want to view my trip routes on an interactive map so that I can visualize where I travelled.
3. As a driver, I want to access my previous trips so that I can review my driving patterns over time.
4. As a driver, I want to replay completed trips on a map so that I can analyze my driving behavior.

3.3 Driving Behavior Monitoring:

1. As a driver, I want the system to detect harsh braking events so that I can improve my driving safety.
2. As a driver, I want the system to monitor my speed during trips so that I can avoid unsafe driving behavior.
3. As a driver, I want dangerous cornering events to be detected so that I can drive more safely.
4. As a driver, I want the system to detect possible crash events so that emergency actions can be triggered quickly. As a driver, I want to receive fatigue and rest-stop alerts during long trips so that I can avoid unsafe driving conditions.

3.4 OBD and Vehicle Diagnostics:

1. As a driver, I want to connect my vehicle to the app using a Bluetooth OBD 2 adapter so that vehicle diagnostics can be monitored.
2. As a driver, I want to monitor my vehicle's health in real time so that I can identify issues early.
3. As a driver, I want to view vehicle error codes so that I can understand potential mechanical problems.
4. As a driver, I want to receive maintenance warnings so that I can prevent serious vehicle failures.

3.5 Fuel Efficiency and Eco-Driving:

1. As a driver, I want the system to estimate fuel usage per trip so that I can better understand my fuel consumption.
2. As a driver, I want personalized eco-driving recommendations so that I can improve fuel efficiency.
3. As a driver, I want to view fuel efficiency trends over time so that I can monitor improvements in my driving habits.

3.6 Safety Scores and Analytics:

1. As a driver, I want to receive a safety score after each trip so that I can evaluate my driving performance.
2. As a driver, I want detailed driving analytics so that I can identify unsafe driving patterns.
3. As a driver, I want the system to classify my driving style so that I can better understand my driving behavior.

3.7 Gamification and Social Features:

1. As a driver, I want to earn badges and achievements so that safe and efficient driving feels rewarding.
2. As a driver, I want to participate in weekly driving challenges so that I stay motivated to improve.
3. As a driver, I want to compare my safety and efficiency scores with other users so that I can track my performance competitively.

4. As a driver, I want to compare my driving performance with users who have similar vehicle models so that the comparisons are more meaningful.

3.8 Emergency and Social Features:

1. As a driver, I want emergency alerts to be sent to trusted contacts during dangerous situations so that help can arrive quickly.
2. As a driver, I want to share my live trip status with my trusted contacts so that they can monitor my safety.

3.9 Fleet Management:

1. As a fleet manager, I want to monitor multiple drivers and vehicles so that I can oversee fleet operations efficiently.
2. As a fleet manager, I want to view fleet-wide safety and fuel analytics so that I can improve overall performance.
3. As a fleet manager, I want to generate reports on driver behavior and fuel efficiency so that I can make informed operational decisions.

3.10 Real-time Notifications:

1. As a user, I want to receive real-time alerts and notifications so that I can respond immediately to important events.
2. As a user, I want my trip and profile data synchronized across devices so that my information is always up to date.

4 Use Cases: Demo 3

4.1 Use Cases

1. User views and possibly reacts to unexpected stop alert

TUCBW: The app detecting a stop, classifying it as unexpected and prompting the user to react during a trip.
TUCEW: The user resolving the unexpected stop event or confirming/not reacting to the dialog, and notifications being sent to trusted contacts.

2. User views and reacts to a long duration alert

TUCBW: The app detecting that a user has been driving for 2 hours continuously, and suggesting nearby rest stops during an active trip.
TUCEW: The user viewing the alert, redirecting to a suggested rest stop and taking a break.

3. View evaluated eco score

TUCBW: The user going on and completing a trip.
TUCEW: The user viewing the system-evaluated eco score for the trip based on fuel usage and acceleration habits.

4. Track user analytics and metrics

TUCBW: The user viewing graphs and analytics on their driving behaviour and fuel efficiency.
TUCEW: The user understanding the correlation between their driving behaviour and fuel efficiency

5. Earn badges for weekly challenges

TUCBW: The driver completing a trip and the system evaluating challenges completed.
TUCEW: The driver seeing their earned badges if any challenges were completed after the trip.

6. Manage account

TUCBW: The user navigating to profile or settings page to manage their account.
TUCEW: The user editing their profile picture or deleting their account.

4.2 Use Case Diagram

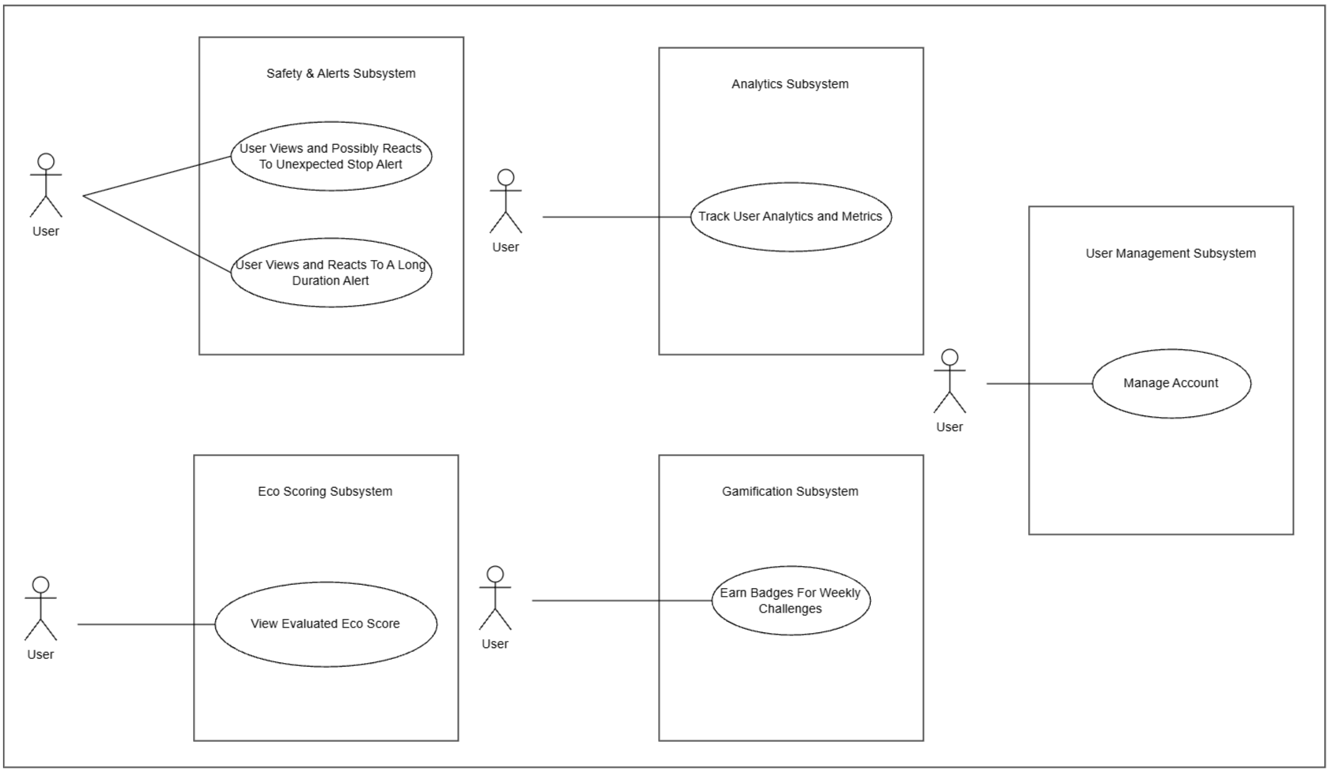

Figure 1: Demo 3 use cases

5 Functional Requirements

5.1 R1: User Management

5.1.1 R1.1 User Registration

R1.1.1: The system should allow users to create accounts using email and password authentication.
R1.1.2: The system should validate user credentials and enforce password security requirements.

5.1.2 R1.2 User Authentication

R1.2.1: The system should allow registered users to securely log into the application.
R1.2.2: The system should allow users to reset forgotten passwords securely.

5.1.3 R1.3 User Profile Management

R1.3.1: The system should allow users to view and update profile information.
R1.3.2: The system should allow users to manage saved contacts.

5.2 R2: Trip Tracking and Monitoring

5.2.1 R2.1: GPS Tracking

R2.1.1: The system should be able to track user trips and routes in real time using GPS services.
R2.1.2: The system should be able to display route information and trip progress on an interactive map.
R2.1.3: The system should be able to monitor trip speed, acceleration, braking, and cornering behavior.

5.2.2 R2.2: Sensor Data Collection

R2.2.1: The system should be able to collect accelerometer, gyroscope, and magnometer data during trips.
R2.2.2: The system should be able to monitor driving events such as harsh braking, rapid acceleration, and dangerous cornering.

5.2.3 R2.3 OBD Integration

R2.3.1: The system should be able to connect to Bluetooth-enabled OBD devices.
R2.3.2: The system should be able to retrieve vehicle diagnostic information, including RPM, coolant temperature, and diagnostic trouble codes.

5.3 R3: Driving Analysis and Safety

5.3.1 R3.1: Driving Behavior Analysis

R3.1.1: The system should be able to analyze driving patterns to identify unsafe driving behavior.
R3.1.2: The system should be able to generate per-trip and overall driver safety scores based on driving behavior.

5.3.2 R3.2: Risk Detection and Alerts

R3.2.1: The system should be able to detect possible crash events and dangerous driving conditions.
R3.2.2: The system should be able to provide real-time alerts for risky driving behavior.
R3.2.3: The system should be able to detect loud impact and screech events using available sensor and microphone data.
R3.2.4: The system should be able to detect fatigue indicators during long-distance trips.
R3.2.5: The system should be able to provide rest-stop alerts for extended driving sessions.

5.3.3 R3.3: Driver Personality Profiling

R3.3.1: The system should be able to classify driver behavior using AI-driven analysis.
R3.3.2: The system should be able to generate personalized driving improvement recommendations.

5.4 R4: Vehicle Health and Efficiency

5.4.1 R4.1: Vehicle Diagnostics

R4.1.1: The system should be able to monitor vehicle health using OBD diagnostic data.
R4.1.2: The system should be able to notify users about potential maintenance issues.

5.4.2 R4.2: Fuel Efficiency Monitoring

R4.2.1: The system should be able to calculate fuel efficiency statistics for trips.
R4.2.2: The system should be able to provide eco-driving recommendations to improve fuel consumption.
R4.2.3: The system should be able to estimate fuel usage per trip using driving behavior and vehicle diagnostic data.

5.5 R5: Gamification and Social Features

5.5.1 R5.1: Achievements and Challenges

R5.1.1: The system should be able to award badges and achievements based on driving performance.
R5.1.2: The system should be able to provide weekly driving challenges for users.

5.5.2 R5.2: Leaderboards

R5.2.1: The system should be able to display leaderboards based on safety, fuel efficiency, and driving consistency scores.
R5.2.2: The system should be able to compare user driving performance with users operating similar vehicle models.

5.5.3 R5.3 Emergency and Social Features

R5.3.1: The system should be able to notify trusted contacts during emergencies.
R5.3.2: The system should be able to allow family and friends to monitor active trips when authorized.

5.6 R6: Fleet Management

5.6.1 R6.1: Fleet Monitoring

R6.1.1: The system should allow fleet managers to monitor multiple drivers and vehicles.
R6.1.2: The system should be able to display fleet-wide trip and safety analytics.

5.6.2 R6.2: Fleet Reporting

R6.2.1: The system should be able to generate reports on driver performance and fuel efficiency trends.

5.7 R7: System Integration and Data Management

5.7.1 R7.1: Cloud Synchronization

R7.1.1: The system should synchronize user and trip data with cloud backend services.

5.7.2 R7.2: Real-time communication

R7.2.1: The system should provide real-time notifications using WebSocket communication.

5.7.3 R7.3: Maps Integration

R7.3.1: The system should be able to integrate with Azure Map services for route visualization and navigation.

5.7.4 R7.4: Data Storage

R7.4.1: The system should be able to securely store user, trip, and vehicle data in cloud databases.

6 Non-Functional Requirements

6.1 Performance

- Real-time sensor and OBD-II data must be processed and reflected in the UI within 2 seconds to avoid degrading the driving experience or navigation responsiveness.
- Trip scoring computations must be completed within 2 seconds of a trip ending.

6.2 Reliability

- The system must achieve a minimum uptime of 99% for backend services.
- The application must handle Bluetooth OBD adapter disconnections gracefully, continuing trip recording using phone sensors alone and notifying the user of the disconnection.
- Background services must recover automatically from crashes without requiring user intervention, considering the app will be used by people driving.

6.3 Maintainability

- A modular architecture should be employed, with clearly separated concerns across sensor processing, OBD communication, trip tracking, gamification, notifications, and authentication.
- New features and/or bug fixes should be deployable within 2 hours.
- Codebase should target and maintain a minimum of 80% automated test coverage.
- The system must include a CI/CD pipeline that runs automated tests and linting on every push, and deploys to production on merge to the main branch.
- Code must follow consistent naming conventions(snake case) and be documented at the module and function level.

6.4 Usability

- The Android application must function correctly on 100% of devices running Android 9 (API level 28) and above.
- The UI shall respond to user interactions within 200 ms during all background operations including sensor sampling, OBD polling, and network sync.
- Driving alerts and notifications shall appear within 3 seconds of detecting an event and shall not require more than one tap to dismiss after the vehicle has stopped.

6.5 Availability

• The API and Database shall maintain 99.5% uptime, excluding scheduled maintenance.
• The system must include a CI/CD pipeline that runs automated tests and linting on 100% of code pushes, and deploys to production on merge to the main branch.
• 95% of public classes and functions shall include documentation comments, and all source code shall comply with the project's agreed naming conventions.

7 Domain Model

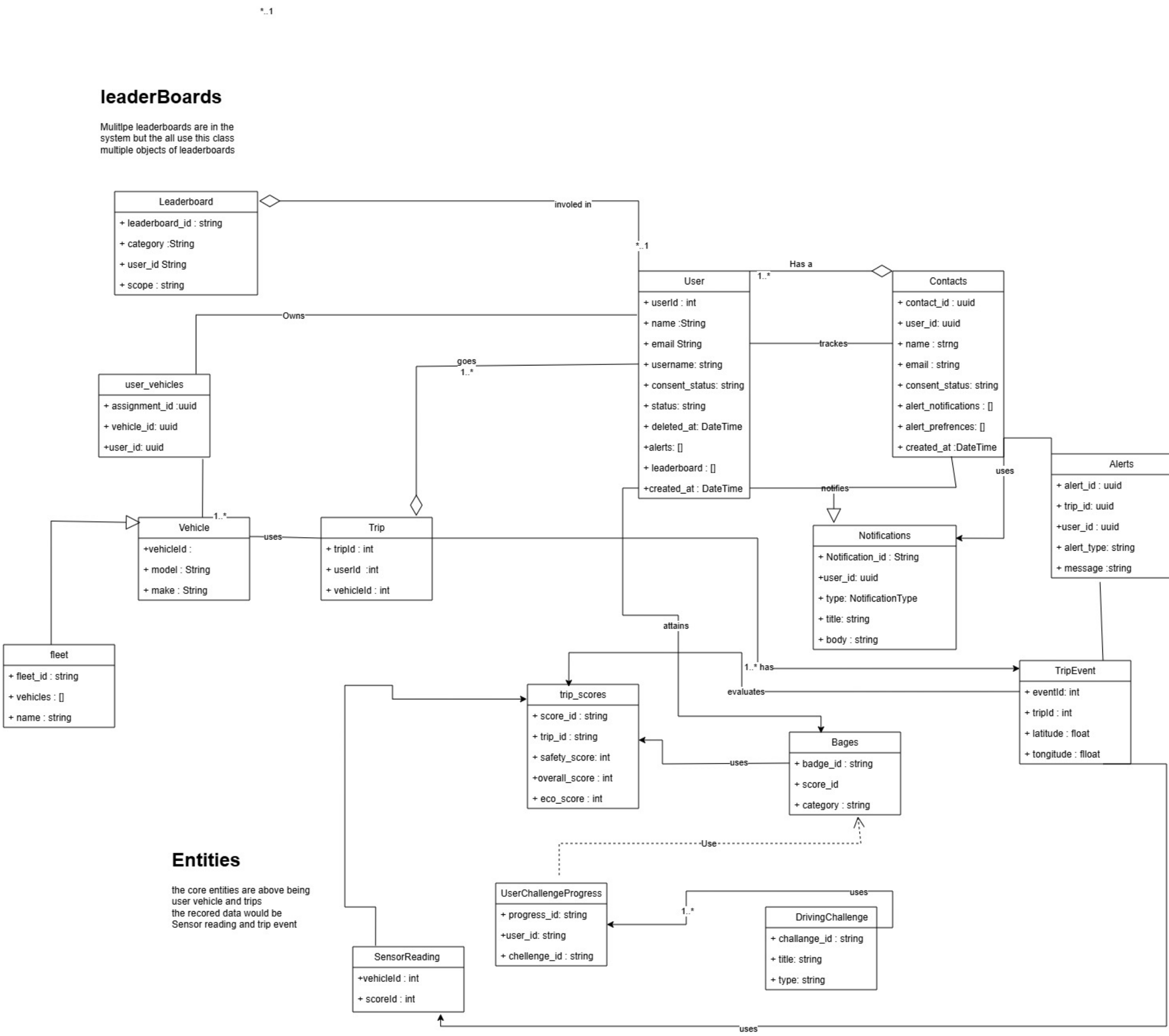

Figure 2: Updated Domain Model